

Algo avancée - Rapport Blobwars

Mehdi DIMASSI

Avril 2025

Table des matières

1	Algorithmes demandés	2
1.1	Fonction d'évaluation	2
1.2	Algorithme glouton	2
1.3	Algorithme minimax	2
1.4	Algorithme Minimax parallèle	2
1.5	Algorithme alpha-beta	3
2	Mesure de performances	4
2.1	Temps moyen de calcul	4
3	Tentatives d'amélioration	5
3.1	Amélioration de la fonction d'évaluation	5
3.2	Exploration itérative	5
3.3	Ordonnancement des coups	6
4	Conclusion	6

1 Algorithmes demandés

1.1 Fonction d'évaluation

Toutes les fonctions demandées nécessitent une fonction d'évaluation afin d'attribuer un score à chaque plateau de jeu possible. Puisqu'au final, c'est le joueur ayant le plus de pions qui l'emporte, une première version de cette fonction peut être formulée ainsi :

$$\text{eval}_{\text{joueur } 0}(x) = \text{nbPions}_{\text{joueur } 0} - \text{nbPions}_{\text{joueur } 1}$$

En effet, plus l'écart de pions entre le joueur et son adversaire est grand, plus il est susceptible de remporter la partie.

Le code associé à cette fonction est présent dans `strategy.cc:estimateCurrentScore()`.

1.2 Algorithme glouton

Un premier algorithme simpliste est l'algorithme glouton, qui énumère tous les coups valides possibles et choisit celui qui mène au plateau ayant le meilleur score selon la fonction d'évaluation.

Le principal avantage de cet algorithme est sa rapidité, car sa complexité est en $O(d)$, avec d le nombre de coups possibles. Cependant, son intérêt reste limité, car ses coups sont facilement prédictibles.

Le code associé à cette fonction est présent dans le fichier `greedy.cc`.

1.3 Algorithme minimax

Cet algorithme explore les coups possibles dans le futur en parcourant l'arbre du jeu jusqu'à une profondeur fixée à l'avance. Il s'apparente davantage au raisonnement d'un joueur humain.

Cependant, l'arbre représentant les états possibles du plateau dans Blobwars croît de manière exponentielle, notamment en milieu de partie où les coups possibles sont nombreux. Par conséquent, la complexité de cet algorithme devient rapidement un facteur limitant, car elle est en $O(d^n)$, avec d le nombre de coups valides et n la profondeur de l'arbre.

On est donc rapidement contraint de choisir une profondeur n relativement faible, car le temps de calcul augmente très vite sinon.

Le code associé à cette fonction implémenté de manière récursive est présent dans le fichier `minimax.cc`.

1.4 Algorithme Minimax parallèle

Une caractéristique avantageuse de Minimax est sa structure naturellement parallèle. En effet, chaque appel récursif sur un coup possible à un niveau donné

de l'arbre est indépendant des autres. Cela permet d'exécuter ces appels en parallèle, en répartissant les branches à explorer sur plusieurs cœurs de calcul.

On note :

— d : le nombre de coups valides par état,

— n : la profondeur de l'arbre.

Le ****travail total**** W est donné par :

$$W = \sum_{i=0}^n d^i = \frac{d^{n+1} - 1}{d - 1}$$

Le ****chemin critique**** D , qui correspond à la profondeur maximale de l'arbre (ou au temps minimal en exécution parallèle parfaite), est de l'ordre de :

$$D = O(n)$$

Ces conditions sont favorables à la parallélisation, car on a :

$$D < \frac{W}{p}$$

avec p le nombre de cœurs disponibles. Cela signifie qu'on peut obtenir une accélération significative.

Comme chaque nœud de l'arbre représente une évaluation ou un appel récursif indépendant, le parallélisme est simple à mettre en œuvre : il suffit de répartir l'exploration des sous-arbres parmi les différents threads.

Le code associé à cette version parallèle de Minimax, implémentée à l'aide de la bibliothèque TBB, est présent dans le fichier `minimax_para.cc`.

1.5 Algorithme alpha-beta

Dans l'arbre Minimax, certaines branches ne rapportent aucune information utile et sont donc inutiles à explorer, comme le montre le cours. En sauvegardant les bornes alpha et bêta tout au long de l'exploration, on peut effectuer des coupures lorsque l'on sait qu'un sous-arbre ne peut pas influencer la décision finale.

Si ces coupures sont faites de manière idéale, par exemple avec l'aide d'un ordonnancement parfait (ce qui est difficile en pratique), on peut réduire la complexité à :

$$O(d^{\frac{n}{2}})$$

Une implémentation séquentielle de cet algorithme est disponible dans le fichier `alpha_beta.cc`.

Parallélisation de l’algorithme alpha-bêta Contrairement à Minimax, la parallélisation de l’algorithme alpha-bêta n’est pas triviale. En effet, les valeurs alpha et bêta calculées dans les branches précédentes sont utilisées pour effectuer les coupures dans les branches suivantes. Ainsi, lorsqu’on parallélise, on perd une partie de l’optimisation obtenue dans l’algorithme séquentiel, car certaines coupures ne peuvent plus être déclenchées à temps.

Une première version parallèle, qui ne parallélise que les sous-arbres du premier niveau (racine), est disponible dans le fichier `alpha_beta_para.cc`.

Une deuxième version plus avancée, implémentant l’heuristique de Feldman-Mysliwicz-Monien du cours, est disponible dans le fichier `feldman.cc`.

2 Mesure de performances

Dans cette section, nous comparons les performances des différents algorithmes implémentés selon le temps de calcul.

Les mesures ont été effectuées à l’aide de la dernière fonction commentée dans le fichier `strategy.cc`. Tous les tests ont été réalisés sur les machines des salles ENSIPC, dans des conditions identiques. La carte standard a été utilisée pour les tests car elle nécessite le plus de calculs.

2.1 Temps moyen de calcul

Tableau des temps moyens de calculs avec un match face à `minimax(3)` ;

Algorithme	Temps moyen (ms)
Glouton	0.2
Minimax profondeur 3	265
Minimax parallèle profondeur 3	76
Alpha-beta profondeur 3	30
Alpha-beta parallèle profondeur 3	5
Alpha-beta Feldman profondeur 3	4.6

Les résultats correspondent aux attentes avec les versions parallèles de alpha beta qui sont les plus rapides.

Cependant, il est important de noter qu’il existe des variations de performances entre les deux variantes de l’algorithme alpha-bêta lorsque la profondeur de l’arbre augmente. Parfois, la version utilisant l’heuristique de Feldman est plus performante, et parfois c’est la version parallèle naïve qui l’emporte.

Cette dispersion peut s’expliquer par l’absence d’un ordonnancement des coups selon leur score estimé. En effet, l’efficacité des coupures alpha-bêta dépend fortement de l’ordre dans lequel les coups sont explorés : un bon ordre permet de déclencher les coupures plus tôt, réduisant ainsi considérablement le nombre de nœuds explorés. Sans cet ordonnancement, les performances peuvent varier de manière imprévisible selon la situation.

3 Tentatives d'amélioration

3.1 Amélioration de la fonction d'évaluation

Pour le moment, notre fonction d'évaluation reste relativement simpliste. Elle repose essentiellement sur le nombre de pions en faveur du joueur, ce qui peut sembler logique puisque le joueur gagnant est celui qui possède le plus de pions.

Cependant, cette approche présente certaines limites. Un coup qui maximise le nombre de pions à une étape donnée n'est pas nécessairement un bon coup à long terme. Il peut conduire à une position désavantageuse ou rendre le joueur vulnérable dans les tours suivants.

Ainsi, il serait judicieux d'envisager une nouvelle fonction d'évaluation, plus sophistiquée, qui représente mieux la probabilité de gagner.

Le jeu *Blobwars* présente des similitudes avec le jeu *Reversi* (aussi appelé *Othello*). Ainsi, il est pertinent d'explorer l'application d'une fonction d'évaluation efficace pour ce dernier au jeu *Blobwars*.

La fonction d'évaluation repose sur cinq paramètres principaux :

- la différence de pions ;
- la capture de coins ;
- la mobilité des pions ;
- les captures potentielles ;
- les pions vulnérables.

Ces cinq paramètres sont pondérés afin de calculer un score global, en fonction de la phase actuelle du jeu (début de partie, milieu de partie, fin de partie).

Plusieurs configurations de pondérations ont été testées à l'aide d'un algorithme génétique, dans le but d'optimiser la fonction d'évaluation.

Une implémentation séquentielle de cette fonction est disponible dans le fichier `strategy.cc` sous le nom de `estimateCurrentScoreImproved`.

Résultats :

Certes, le temps de calcul a légèrement augmenté, mais à profondeur d'exploration égale, la nouvelle fonction d'évaluation remporte 80% des parties. De plus, elle parvient parfois à gagner même en explorant à une profondeur moindre que son adversaire.

3.2 Exploration itérative

Un problème de l'implémentation actuelle réside dans le fait que la profondeur de l'arbre est fixée dès le départ. En conséquence, l'algorithme termine parfois l'exploration bien avant la fin du temps imparti, tandis qu'à d'autres moments, il n'a pas le temps de la terminer. Ce comportement dépend fortement de la phase du jeu (début, milieu ou fin), qui influence la densité et la complexité des coups possibles.

Une solution consiste à adopter une stratégie d’exploration itérative, où la profondeur est augmentée progressivement (par exemple, de 1 à chaque itération), jusqu’à atteindre la limite de temps disponible. Cela permet de toujours utiliser au mieux le temps alloué, tout en garantissant que les meilleures décisions sont prises avec les ressources disponibles.

Il est important de noter que cette approche n’augmente pas réellement la complexité algorithmique dans le pire des cas, puisqu’on effectue les mêmes évaluations que dans une exploration classique à profondeur maximale. Cependant, elle améliore significativement l’efficacité en pratique, en s’adaptant dynamiquement aux situations de jeu.

Analyse de la complexité

L’exploration itérative effectue successivement les recherches aux profondeurs $1, 2, \dots, n$. Si d est le facteur de branchement moyen et n la profondeur maximale atteinte, alors le nombre total de nœuds explorés est :

$$\sum_{i=1}^n d^i = d + d^2 + d^3 + \dots + d^n = \frac{d^{n+1} - d}{d - 1} = \mathcal{O}(d^n)$$

3.3 Ordonnancement des coups

Si l’on parvient à jouer les meilleurs coups dès le début de l’exploration, cela permet d’augmenter significativement le nombre de coupures effectuées par l’algorithme alpha-bêta, et donc de réduire le nombre de nœuds explorés.

Cependant, effectuer un ordonnancement précis des coups à chaque tour en évaluant systématiquement tous les coups possibles avec la fonction d’évaluation serait extrêmement coûteux en temps de calcul.

Une alternative plus légère consiste à simplement essayer en priorité le meilleur coup identifié à la profondeur précédente. Cette méthode, bien que moins optimale qu’un ordonnancement complet, reste beaucoup plus rapide à exécuter et permet déjà une amélioration notable du pruning.

Résultats

Des tests effectués sur plusieurs parties montrent un gain moyen de 20 à 50 ms par tour de calcul.

4 Conclusion

Les deux algorithmes testés — *Alpha-Bêta parallèle à la racine* et l’algorithme de *Feldman* — présentent des performances relativement similaires. L’intégration de l’exploration itérative ainsi que l’ordonnancement du premier coup ont permis d’améliorer sensiblement l’efficacité de ces deux approches, en réduisant le temps de calcul et en augmentant la profondeur explorée.

Cependant, la nouvelle fonction d'évaluation proposée n'a pas apporté de gain clair dans sa version actuelle. Son taux de victoire reste proche de 50%, ce qui indique qu'elle ne surpasse pas de manière significative la version précédente. Cette limitation s'explique en partie par le fait que l'algorithme génétique utilisé pour ajuster les poids n'a pu être simulé qu'à une profondeur fixe de 3, en raison de contraintes de temps de calcul.

Or, la version finale de notre moteur repose sur une exploration itérative avec des profondeurs variables. Ainsi, les poids optimisés à profondeur fixe ne sont probablement pas adaptés à cette nouvelle dynamique, ce qui limite l'efficacité de la fonction d'évaluation. Une optimisation plus poussée, prenant en compte l'exploration itérative, pourrait offrir de meilleurs résultats à l'avenir.